

Instructor and Self-Study Solutions Companion

This companion keys every exercise in Volume III. A concise rubric is used where an exercise deliberately admits several correct constructions. The companion is an internal pedagogical artifact and does not constitute independent mathematical review.

Contents

1	Solutions to Chapter 1	1
2	Solutions to Chapter 2	3
3	Solutions to Chapter 3	5
4	Solutions to Chapter 4	7
5	Solutions to Chapter 5	9
6	Solutions to Chapter 6	11
7	Solutions to Chapter 7	13
8	Solutions to Chapter 8	15
9	Solutions to Chapter 9	17
10	Solutions to Chapter 10	19
11	Solutions to Chapter 11	21
12	Solutions to Chapter 12	23
13	Solutions to Chapter 13	25
14	Solutions to Chapter 14	27

Chapter 1

Solutions to Chapter 1

1.1

Solution / rubric. $\lambda x:P.x : P \rightarrow P$ by variable then implication introduction.

1.2

Solution / rubric. $\rightarrow E$: from $\Gamma \vdash f : A \rightarrow B$ and $\Gamma \vdash a : A$, infer $\Gamma \vdash fa : B$.

1.3

Solution / rubric. $\Gamma \vdash A$ hides the selected witness; $\Gamma \vdash t : A$ records it. Hiding a witness does not equate all witnesses.

1.4

Solution / rubric. Use $\lambda x:P.\lambda y:Q.x : P \rightarrow (Q \rightarrow P)$.

1.5

Solution / rubric. One term is $\lambda f.\lambda g.\lambda x.g(fx)$ with the declared types; it is function composition in the stated argument order.

1.6

Solution / rubric. Type $x : P$, then $fx : Q$, then $g(fx) : R$; both application heads must have matching arrow domains.

1.7

Solution / rubric. Exact: the premises/conclusion of $\rightarrow I / \rightarrow E$ determine lambda/application. Difference: a derivation tree and a compact bound term have different bureaucracy.

1.8

Solution / rubric. The chapter relates logical and type constructors; it does not quotient data-bearing types to truth values.

1.9

Solution / rubric. Induct on typing; variable and application are immediate, while abstraction may require alpha-renaming before weakening.

1.10

Solution / rubric. Induct on the final natural-deduction rule; assumption, $\rightarrow I$, and $\rightarrow E$ are exhaustive in the implication fragment.

1.11

Solution / rubric. A correct design parses right-associative arrows into the checker AST and rejects dangling arrows, unbalanced parentheses, and reserved tokens as atoms.

1.12

Solution / rubric. With rule-node/AST-constructor size, the selected assignment can be size-preserving; counting discharge/context annotations introduces administrative overhead. Credit requires the metric first.

Chapter 2

Solutions to Chapter 2

2.1

Solution / rubric. $(\lambda x:P.x)u \longrightarrow u$.

2.2

Solution / rubric. Premises: $\Gamma, x:A \vdash t : B$ and $\Gamma \vdash u : A$; conclusion $\Gamma \vdash t[u/x] : B$.

2.3

Solution / rubric. Rename when a binder would capture a free variable of the replacement.

2.4

Solution / rubric. The term reduces first to $\lambda x:P.(\lambda z:P.z)x$ and then to $\lambda x:P.x$, preserving type $P \rightarrow P$.

2.5

Solution / rubric. For $(\lambda x:P.\lambda y:Q.x)y$, rename the inner binder before substitution, yielding $\lambda y_1:Q.y$.

2.6

Solution / rubric. Invert application and abstraction typing, then apply substitution to the body.

2.7

Solution / rubric. Proof detour elimination substitutes the supplied derivation for a discharged assumption; beta reduction performs the corresponding capture-avoiding term substitution under the assignment.

2.8

Solution / rubric. The reduction relation permits compatible redexes; a deterministic evaluator chooses one next redex and therefore one path.

2.9

Solution / rubric. Alpha-rename the binder fresh for u , weaken u into the extended context, apply the induction hypothesis, and reintroduce the lambda.

2.10

Solution / rubric. Use the induction hypothesis on the function subterm and reapply application typing.

2.11

Solution / rubric. A stable path alphabet such as `fn, arg, body` is sufficient; recheck that the recorded path identifies a redex and preserves the invariant type.

2.12

Solution / rubric. Strong normalization forbids every infinite reduction sequence; termination of one strategy, and especially termination on fixtures, is strictly weaker.

Chapter 3

Solutions to Chapter 3

3.1

Solution / rubric. $\times I$ builds a pair; $\times E_1$ and $\times E_2$ project its components.

3.2

Solution / rubric. $\pi_2 \langle a, b \rangle \longrightarrow b$.

3.3

Solution / rubric. The pair retains both witnesses and therefore supports two computationally distinct projections.

3.4

Solution / rubric. $\lambda x: P. \langle x, x \rangle : P \rightarrow (P \times P)$.

3.5

Solution / rubric. $\lambda p: P \times Q. \langle \pi_2 p, \pi_1 p \rangle : (P \times Q) \rightarrow (Q \times P)$.

3.6

Solution / rubric. Reduce inside the first component if required, then the outer projection; final result is the normalized first component.

3.7

Solution / rubric. Both eliminate a compound object by selecting a field; the logical reading is scoped to conjunction evidence.

3.8

Solution / rubric. Canonical forms follows from the declared value grammar plus typing inversion; it says nothing about untyped host data.

3.9

Solution / rubric. Substitution distributes recursively through pair construction and projection; invoke the induction hypothesis on the component subderivations.

3.10

Solution / rubric. Invert the pair typing and select the corresponding component type for each projection reduction.

3.11

Solution / rubric. Reject projection of non-products, ill-typed pair components, and mismatched claimed projection types.

3.12

Solution / rubric. Product eta $p \equiv \langle \pi_1 p, \pi_2 p \rangle$ is an equational law here, not a designated one-step evaluator rule.

Chapter 4

Solutions to Chapter 4

4.1

Solution / rubric. $+I_1$ maps $a : A$ to $\text{inl}(a) : A + B$; $+I_2$ is symmetric.

4.2

Solution / rubric. The eliminator must have one well-defined result type regardless of which injection is observed.

4.3

Solution / rubric. A case over $\text{inr}(b)$ contracts to the right branch with b substituted for its binder.

4.4

Solution / rubric. $\lambda x:P. \text{inl}(x) : P \rightarrow (P + Q)$.

4.5

Solution / rubric. Case on the input; map left p to $\text{inr}(p)$ and right q to $\text{inl}(q)$.

4.6

Solution / rubric. Given $f : P \rightarrow R$ and $g : Q \rightarrow R$, case on $s : P + Q$ and return fp or gq ; both have type R .

4.7

Solution / rubric. A Boolean flag contains a tag only; a sum contains a tag plus evidence/-payload for the chosen branch.

4.8

Solution / rubric. The case consumer can observe which proof constructor was used and may compute differently.

4.9

Solution / rubric. Substitute recursively in the scrutinee and both branches, alpha-renaming branch binders when necessary.

4.10

Solution / rubric. Invert the right injection and branch typing, then use substitution to type the selected reduct.

4.11

Solution / rubric. AST fields: scrutinee, left binder/body, right binder/body; type both bodies under extended contexts and require equal codomains.

4.12

Solution / rubric. A sum eta principle belongs to the equational layer unless explicitly oriented as a reduction; it is not one of the chapter's principal operational steps.

Chapter 5

Solutions to Chapter 5

5.1

Solution / rubric. $\star : 1$.

5.2

Solution / rubric. $\mathbf{0}$ represents an unconstructible proposition/type in the core, so it has no constructor.

5.3

Solution / rubric. For closed well-typed t , either t is a value or the chosen evaluator can take a step.

5.4

Solution / rubric. $\lambda e:\mathbf{0}. \text{abort}_P(e) : \mathbf{0} \rightarrow P$.

5.5

Solution / rubric. Arrow/lambda; product/pair; sum/injection; unit/ $\star$; empty/no closed value.

5.6

Solution / rubric. Use progress on the function; if it steps, lift the step through application; if it is a value, canonical forms expose a lambda.

5.7

Solution / rubric. Ex falso is an eliminator conditioned on evidence of **0**; it introduces no such evidence.

5.8

Solution / rubric. Type safety excludes stuck well-typed states; evaluator termination concerns one strategy; strong normalization excludes all infinite reduction sequences.

5.9

Solution / rubric. Progress the scrutinee; if it is a value, sum canonical forms selects an injection and hence a case redex.

5.10

Solution / rubric. Apply the induction hypothesis to the stepped subterm and reapply the surrounding typing rule.

5.11

Solution / rubric. Generate syntax independently of the expected progress result, then filter/type-check; include malformed terms as negative controls rather than preconstructing only redexes.

5.12

Solution / rubric. Fundamental lemma: a well-typed term instantiated by a substitution mapping each context variable to a reducible term yields a reducible term at the judgment type.

Chapter 6

Solutions to Chapter 6

6.1

Solution / rubric. $\Pi(x : A).B(x)$ is a construction producing evidence of $B(a)$ for every supplied $a : A$.

6.2

Solution / rubric. $\Sigma(x : A).B(x)$ packages one witness a and evidence of $B(a)$.

6.3

Solution / rubric. Application substitutes the supplied a for x in the codomain, yielding $B(a)$.

6.4

Solution / rubric. For a two-element domain, a term $f : \Pi(b : \text{Bit}).B(b)$ supplies both indexed cases through one dependent function.

6.5

Solution / rubric. A package $\langle a, b \rangle$ has witness $a : A$ and evidence $b : B(a)$.

6.6

Solution / rubric. If B is constant in x , $\Pi(x : A).B$ reduces to the ordinary nondependent function shape $A \rightarrow B$.

6.7

Solution / rubric. Like an indexed record, the type of the second field is determined by the first field's value.

6.8

Solution / rubric. Erasing the witness can destroy the value that downstream computation needs, so erasure requires a relevance theorem/policy.

6.9

Solution / rubric. Read each dependent introduction/elimination premise propositionally and map it to the matching constructor/eliminator; no additional cases are needed for the displayed fragment.

6.10

Solution / rubric. The motive/branch must remain well formed after capture-avoiding substitution of the witness and dependent evidence.

6.11

Solution / rubric. Enumerate the finite witness domain, recompute the registered predicate for each package, and reject witnesses outside the domain or evidence with the wrong indexed claim.

6.12

Solution / rubric. Substitution, preservation, progress/canonical forms, and normalization all require fresh dependent proofs before transfer to a richer calculus.

Chapter 7

Solutions to Chapter 7

7.1

Solution / rubric. Cut composes $\Gamma \Rightarrow A$ and $\Delta, A \Rightarrow C$ into $\Gamma, \Delta \Rightarrow C$.

7.2

Solution / rubric. A is the cut formula: the explicit intermediate proposition introduced by one premise and consumed by the other.

7.3

Solution / rubric. The first derivation supplies an intermediate object that the second derivation uses; eliminating the intermediate composes the proofs directly.

7.4

Solution / rubric. Any correct sequent derivation that derives an implication then cuts it into a consumer earns credit if the end sequent is preserved.

7.5

Solution / rubric. Under the ND/term translation, substitute the proof/term for the open assumption corresponding to the cut formula.

7.6

Solution / rubric. One standard measure is lexicographic: cut-formula complexity first, then proof height or sum of premise heights.

7.7

Solution / rubric. Both remove an intermediate proof detour, but their syntax and local transformations are different and related by translation.

7.8

Solution / rubric. Finite traces cover only enumerated proof trees; Hauptsatz quantifies over every derivation in the calculus.

7.9

Solution / rubric. Expose the final implication-right and implication-left rules and replace the cut on $A \rightarrow B$ by cuts on structurally smaller components/premises.

7.10

Solution / rubric. Commute a cut upward past an inference in which the cut formula is not principal, preserving the end sequent while reducing height.

7.11

Solution / rubric. Store rule tag, premise trees, end sequent, and optional cut formula; validate the conclusion from the rule and premises.

7.12

Solution / rubric. Cut admissibility requires an induction proving every derivation with cut can be transformed into a cut-free derivation of the same end sequent.

Chapter 8

Solutions to Chapter 8

8.1

Solution / rubric. $\neg A$ abbreviates $A \rightarrow \mathbf{0}$.

8.2

Solution / rubric. One must supply either a left injection with A evidence or a right injection with $\neg A$ evidence.

8.3

Solution / rubric. Because extension w_1 forces P but not $\mathbf{0}$, the implication $P \rightarrow \mathbf{0}$ fails at w_0 .

8.4

Solution / rubric. w_1 forces P , so it forces $P \vee \neg P$ by the left disjunct.

8.5

Solution / rubric. If an atomic proposition is forced at w , it remains forced at every extension $v \geq w$.

8.6

Solution / rubric. Any finite monotone frame that falsifies the selected schema is acceptable if all forcing clauses are checked.

8.7

Solution / rubric. The semantic countermodel yields syntactic nonderivability only after applying soundness: derivable formulas are valid in every model.

8.8

Solution / rubric. Failure of constructive derivability means the rules do not construct the formula; it is not the same assertion as classical falsity under a valuation.

8.9

Solution / rubric. Induct on formula structure; for implication use transitivity of the accessibility relation to propagate the universal extension condition.

8.10

Solution / rubric. Check root nonforcing of P , root nonforcing of $\neg P$, and the disjunction clause; verify monotonic atomic forcing.

8.11

Solution / rubric. Require a finite preorder, monotone atomic forcing, valid formula syntax, and referenced worlds/propositions to exist.

8.12

Solution / rubric. Excluded middle adds a proof principle not generated by witness-producing introduction rules, altering normalization/extraction interpretations even when classical truth tables are familiar.

Chapter 9

Solutions to Chapter 9

9.1

Solution / rubric. A continuation is a function describing how the rest of the computation consumes a produced value.

9.2

Solution / rubric. Fixing R gives every continuation a common target and makes the translation typing equations determinate.

9.3

Solution / rubric. With $R = \mathbf{0}$, a CPS result has a double-negation-like outer shape.

9.4

Solution / rubric. Under the stated rules, translate identity so it receives a continuation and passes its argument to that continuation.

9.5

Solution / rubric. Each fresh continuation accepts exactly the translated intermediate value and returns the common answer type R .

9.6

Solution / rubric. Evaluate both forms on the same closed fixture; the chapter requires equal observed result for the retained examples, not general contextual equivalence.

9.7

Solution / rubric. A translation provides a defined structure-preserving map; that is more precise than analogy but still relates distinct source and target syntaxes.

9.8

Solution / rubric. Control can make continuation/context behavior computationally relevant, so constructive erasure/extraction theorems need new hypotheses.

9.9

Solution / rubric. Assume the translated body is well typed under the translated argument; build the target function and pass it to the current continuation.

9.10

Solution / rubric. Translate function and argument through fresh continuations, type each bound intermediate, then apply the translated function and outer continuation.

9.11

Solution / rubric. Generate fresh names outside the union of source free variables and names introduced earlier by the translation; alpha-rename source binders if needed.

9.12

Solution / rubric. One must prove a normalization theorem for the actual extended control calculus, including its new reduction rules; CH-0 normalization cannot simply be cited.

Chapter 10

Solutions to Chapter 10

10.1

Solution / rubric. Proof evidence is relevant when a context can observe which inhabitant was supplied and change retained behavior.

10.2

Solution / rubric. Left and right injections of a sum can select different case branches.

10.3

Solution / rubric. No. CH-0 treats proof terms as ordinary computationally meaningful inhabitants unless a separate quotient/erasure rule is stated.

10.4

Solution / rubric. $\text{inl}(a)$ and $\text{inr}(b)$ inhabit the same sum type but a case context can return different results.

10.5

Solution / rubric. Any argument used only in checked ghost/erasable positions is acceptable under the toy policy.

10.6

Solution / rubric. A value that controls a retained branch must remain; erasing it can change observable output.

10.7

Solution / rubric. Proof irrelevance is a semantic/type-theoretic equality principle; dead-code elimination is a compiler transformation justified by a program analysis.

10.8

Solution / rubric. Extraction may keep only a computational projection of the proof term, so the source proof and deployed program need not be byte- or syntax-identical.

10.9

Solution / rubric. Apply the induction hypotheses to function and argument; since ordinary application is retained, reconstructed application preserves the retained behavior.

10.10

Solution / rubric. The checker must prove the ghost-bound value has no dependency path to retained constructors, guards, outputs, or effects.

10.11

Solution / rubric. Mark a branch-controlling Boolean/tag as ghost; a sound checker must reject the erasure because outputs differ.

10.12

Solution / rubric. Proof irrelevance can simplify proposition fields used solely for certification; the intentional cost is losing distinctions among their inhabitants.

Chapter 11

Solutions to Chapter 11

11.1

Solution / rubric. Evaluation runs an already executable term; extraction transforms a proof-bearing artifact into the program intended for execution.

11.2

Solution / rubric. The selected existential extractor retains the witness and drops the already-checked certificate.

11.3

Solution / rubric. Replay verifies that the exact retained witness still satisfies the registered decidable specification.

11.4

Solution / rubric. Example: witness 2 for $n^2 = 4$ over the finite domain, paired with a checked certificate.

11.5

Solution / rubric. The runtime needs the witness; the proof certificate may be dropped only after the pipeline has checked it under the stated policy.

11.6

Solution / rubric. Pair witness 3 with a forged certificate claiming $3^2 = 4$; the checker must reject it before extraction.

11.7

Solution / rubric. Proof irrelevance concerns equality/observability of proofs; extraction is a transformation deciding what evidence is retained at runtime.

11.8

Solution / rubric. Classical axioms, opaque constants, effects, unsafe casts, or external oracles are acceptable examples.

11.9

Solution / rubric. Acceptance establishes $\text{predicate}(a)$; extraction returns that same a ; replay recomputes the same predicate, hence succeeds.

11.10

Solution / rubric. A stronger theorem would quantify over all well-typed sources and prove compilation/erasure semantic preservation, including every source construct and runtime rule.

11.11

Solution / rubric. Checker validates proof-bearing input; extractor transforms only validated artifacts; replay independently checks the retained result/specification pair.

11.12

Solution / rubric. Execution validates one pipeline/output relation; it cannot detect that the formal source statement was an incorrect encoding of the intended informal theorem.

Chapter 12

Solutions to Chapter 12

12.1

Solution / rubric. Typing says which instances are legal; parametricity adds uniform relational behavior across those instances.

12.2

Solution / rubric. With no type-specific operations or constants of unknown α , the only generally available α is the supplied argument, under the usual purity/parametricity assumptions.

12.3

Solution / rubric. It is exhaustive evidence over the declared finite fixture universe, not a proof of the abstraction theorem.

12.4

Solution / rubric. For identity, every related pair $(a, b) \in R$ maps to the same pair (a, b) , which remains in R .

12.5

Solution / rubric. A type-specific branch that flips one finite carrier while leaving the other unchanged will violate a suitably chosen relation.

12.6

Solution / rubric. Runtime type inspection supplies representation-specific information that ordinary parametric reasoning assumes unavailable.

12.7

Solution / rubric. Both prevent clients/implementations from exploiting hidden representation details beyond the declared interface.

12.8

Solution / rubric. Purity and relational parametricity are two central hypotheses; termination/effect assumptions may also matter for the exact theorem.

12.9

Solution / rubric. The finite proposition follows by enumerating every declared related input pair and checking target-relation membership of the outputs.

12.10

Solution / rubric. Arrow relations relate f and g when they map every pair of related inputs to a pair of outputs related at the codomain relation.

12.11

Solution / rubric. Generate nonempty, empty, total, singleton, and structured relations deliberately so the suite cannot pass solely because every relation is vacuous.

12.12

Solution / rubric. Independent type instantiations test samples; parametricity quantifies over relations connecting instantiations and thereby constrains cross-instance behavior.

Chapter 13

Solutions to Chapter 13

13.1

Solution / rubric. Elaboration fills in/constructs explicit proof terms; kernel checking independently validates the explicit object against the small trusted rules.

13.2

Solution / rubric. The kernel need not trust an elaborator success flag; it receives only explicit syntax/context/type data that it reparses and checks.

13.3

Solution / rubric. At minimum the kernel implementation, parser, runtime/hardware, foundational rules, and correctness of the formalized statement remain trusted obligations.

13.4

Solution / rubric. Change the claim for $\lambda x:P.x$ from $P \rightarrow P$ to $Q \rightarrow Q$; independent inference rejects it.

13.5

Solution / rubric. If elaborator and kernel share a buggy parse result, the apparent independent check may inherit the same interpretation error.

13.6

Solution / rubric. Formalize the unintended theorem $P \rightarrow P$ when the prose intended a stronger claim; the proof can be fully correct for the wrong statement.

13.7

Solution / rubric. Both carry an explicit object that a smaller verifier can check without trusting the producer's entire reasoning process.

13.8

Solution / rubric. A smaller checker reduces the trusted surface and improves auditability; bugs, foundational assumptions, and specification errors remain possible.

13.9

Solution / rubric. By interface definition the kernel discards producer authority and recomputes derivability from explicit inputs, so acceptance is a function of kernel rules and object only.

13.10

Solution / rubric. Parser correctness is a separate obligation: the checked AST must represent the supplied serialized certificate without ambiguity or producer-controlled reinterpretation.

13.11

Solution / rubric. Use explicit context, term, claimed type, and structured diagnostics; avoid accepting opaque producer-side validation tokens.

13.12

Solution / rubric. Credit requires identifying informal specification, formal statement, elaboration, kernel, compiler/runtime, OS/hardware; formal proof primarily addresses derivability inside the formal statement/calculus.

Chapter 14

Solutions to Chapter 14

14.1

Solution / rubric. A service can be correct precisely by remaining available for further legal messages; termination may mean service failure.

14.2

Solution / rubric. Examples: who sends first, allowed message order, peer duality, resource linearity, or liveness.

14.3

Solution / rubric. A handler may terminate on each message while the sequence of messages violates the protocol transition relation.

14.4

Solution / rubric. Example: states `start` and `await_ack`; `hello` moves to `await_ack` and `ack` returns to `start`.

14.5

Solution / rubric. An `ack` in the initial state violates the request-before-response ordering.

14.6

Solution / rubric. Safety: every observed transition is protocol-legal. Liveness: under stated scheduling/peer assumptions an enabled conversation eventually makes required progress.

14.7

Solution / rubric. Session fidelity concerns conformance to typed communication structure; distributed correctness additionally depends on failures, scheduling, networks, fairness, persistence, and other system assumptions.

14.8

Solution / rubric. Strongest surviving form: exact rule/term and principal normalization correspondences in scoped constructive calculi, with translations or added hypotheses required beyond them.

14.9

Solution / rubric. Use two terminating local handlers plus an illegal initial `ack`; local termination holds while the trace checker rejects the first transition.

14.10

Solution / rubric. Global progress needs complementary endpoint typing plus assumptions about channel semantics, scheduling/fairness, recursion, and absence/handling of failures appropriate to the theorem.

14.11

Solution / rubric. Expose endpoint role, protocol state, permitted send/receive labels and payloads, transition targets, resource usage, and recursion/termination points.

14.12

Solution / rubric. Minimum vocabulary: channels/endpoints, duality, send/receive, branch/select, protocol state, linear/resource discipline, recursion, operational semantics, fidelity, progress, and deadlock hypotheses.

Scope note

The solutions support internal pedagogical closure. They neither replace the theorem audit nor satisfy Gate 8 independent mathematical review.